

I. INTRODUCTION

Artificial Neural Networks (referred to as ANNs) have become a very important aspect in the world of machine learning due to their many implications. Learning off of examples over time, ANNs are able to perform tasks and make decisions without any task-specific rules, meaning they are not constrained to any specific task at hand, but can easily be adapted to perform and complete any problem that needs solving. Thus, these neural nets have a wide range of applicability, as long as the input data is appropriately encoded, but that goes without saying for the vast amount of machine learning techniques.

Thus, it was required in this practical to implement an ANN with back-propagation, in order to propose objects for a Rock-Paper-Scissors (RPS) agent to play in a match, and over multiple iterations determine the best move to play concerning the inputted data set. But what is also important in this practical, is to not create an agent which is over-trained and not able to generalize, but one that can successfully perform using data other than just the example training data.

II. THEORETICAL BACKGROUND

There are multiple aspects of a ANN that constitute its structure, and ultimately its success as a machine learning algorithm. An important understanding of these aspects enables efficient implementation and ultimately an understanding of how different factors affect the operation of an ANN. Such aspects include the structure and hyper-parameters, training linked to forward- and backward-propagation, and lastly stopping conditions which confirms the success of said ANN.

A. Layers of nodes within an Artificial Neural Network

An ANN consists of arrays of input nodes, hidden layers and output nodes, for which establish the basic structure of an ANN.

1) *How inputs are represented:* For the context of this practical, inputs will be encoded with one-hot encoding. This means that every possible move has a specific set of numbers correlated to it, as can be found in the table below:

Move	Encoding
R	1 0 0
P	0 1 0
S	0 0 1

TABLE I: One-hot encoding used for inputs

This encoding will be a 4 x 3 array sent directly to the input of the neural network.

2) *How outputs are represented:* In the same way that inputs are used with one-hot encoding, the outputs will behave similarly, with a small change. When looking at a rock paper scissors game, a scale of [-1,1] would suffice, where a value being 1 represents the highest confidence/most optimal move and -1 representing the worst move for an expected output. This would be a good method to train a neural network, since it should not only focus on winning (playing the best move possible), it should also focus on not losing. For example, when an opponent only plays rock, and you input an expected outcome of paper, the neural network will think that rock and scissors perform equally. This however is not the case. When you play rock, you end up drawing, whereas if you had played scissors, you would have lost the round. Below is a table describing the output encoding that will be used. These values

Move	Encoding
R	1 -1 0
P	0 1 -1
S	-1 0 1

TABLE II: One-hot encoding used for outputs

might seem confusing, but here is an example to clear it up. When the output is 'R', the move the opponent is expected to play is 'S' (We are trying to predict their move). So by playing paper we would lose (which is why paper is -1). playing 'S' would thus lead to a draw (0).

3) *The input Layer:* Input nodes represent input criterion based on the data being sent into the ANN, and are required to be appropriately encoded in order to represent the problem correctly, as well as set up a base for finding an appropriate solution. Thus in the scenario of a RPS runner, input nodes will represent appropriate moves that can be played, namely being rock, paper or scissors.

4) *The hidden Layer:* The hidden layer is a layer between the input and output layer. It adds a layer of complexity and allows for a more refined solution. There can be any number of hidden layers, and the amount of nodes each hidden layer has can also be specified. Hidden layers form the intermediary stages between input and output, and are responsible for learning the intricate structures in data, as a simple structure only consisting of input and output nodes are not enough to accurately determine the complex patterns in a problem space. Thus these hidden layers consist of deeper parameters, for example the activation function. The goal for the hidden layers is to essentially "piece together" the solution. The hope is for the hidden layer within the ANN layer to recognize patterns and predict the output of the system.

5) *The output Layer:* As input nodes represent moves played in a RPS game, subsequently the output nodes will be used to determine the solution to these games, or rather the move that most likely needs to be played in order to beat a semi-predictable opponent. The output nodes represent all the possible solutions. The one-hot encoded outputs will be used to choose which move to play, where the highest value will be selected as the chosen move

B. Weights and biases

A very important aspect in the ability for a ANN to learn, are the links between all the nodes, and the subsequent weight between these links. All nodes in a layer are linked to every other node in subsequent and consequent adjacent layers in a neural network, with these links furthermore carrying a weight that are applied to subsequent and consequent values that are passed between outputs and inputs of adjacent nodes between layers. It essentially tells the network how important a node is to another node. Biases are values that will be added to the weighted input. It acts as a way to bypass any weights and inputs to ensure that the neuron/node is activated. Biases allow nodes/neurons to activate in response to various input conditions. They shift the activation function across the plane either to the left or the right.

The goal of a neural network is to ultimately find the optimal combination of weights and biases in order to correctly find the solution of an entire problem space. And this is what makes ANN so useful, as they can be changed in order to model any problem space, and efficiently find the wanted solutions.

C. The Activation function

The activation function constitutes the process in which an input signal is transformed of the node and then passed to the output of node and subsequently to the input of the next node. The goal of the activation function is to limit the outputs and inputs into a finite space. An example of an activation function is the sigmoid function, which limits the outputs of a variable between 0 and 1. Activation functions form an integral building block of ANN's as they enable them to ultimately learn complex patterns in data, but this will be explained in more detail later when looking at the implementation method of a neural network. The graph of the sigmoid function can be seen below, showing how the output levels between 0 and 1 for any given input.

Although the sigmoid is a very useful function, its use as an activation has declined over the past years because of more versatile activation functions. An example of a different activation function is tanh, which divides the output spectrum to a range between -1 and 1. More information on this will be given in the hyperparameter section of the report.

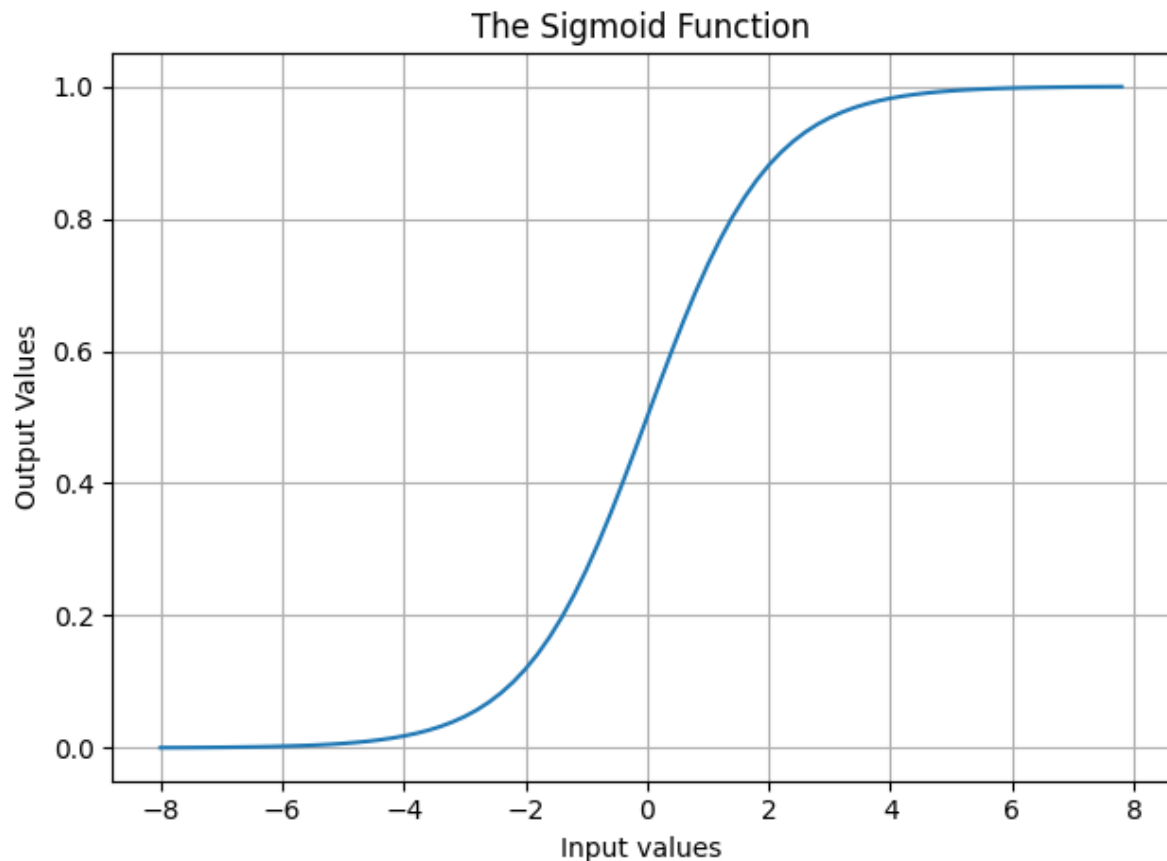


Figure 1: The Sigmoid function

D. The basics of the Neural Network

Putting the input, hidden and output layers together and connecting them with different weights and biases through an activation function, the Neural network is now almost complete. An example of a neural network with 12 inputs can be seen in the figure below. This shows the optimal output when playing against an opponent that only plays rock.

The white lines between each node represents the weight and bias going through to that node. The numbers inside the nodes/neurons is the input/output value of that specific node. There are some annotations that provides more information on the type of encoding (one-hot encoding in this example photo).

E. How the Neural network learns

Now that the structure of an ANN is clearly defined, a crucial understanding of the two main processes concerned with the operation of ANN is required, namely being forward-propagation and backward-propagation.

1) Forward propagation: The ANN begins with performing forward-propagation which entails input data being fed through the network, in a forward direction in order to generate an output. The process of forward-propagation is directly linked to connections between nodes and the implications of activation functions and weights between adjacently linked nodes.

Forward propagation essentially represents a chain of links, starting at input nodes connected to hidden layers and finally connected to output nodes, and the passing of data through these

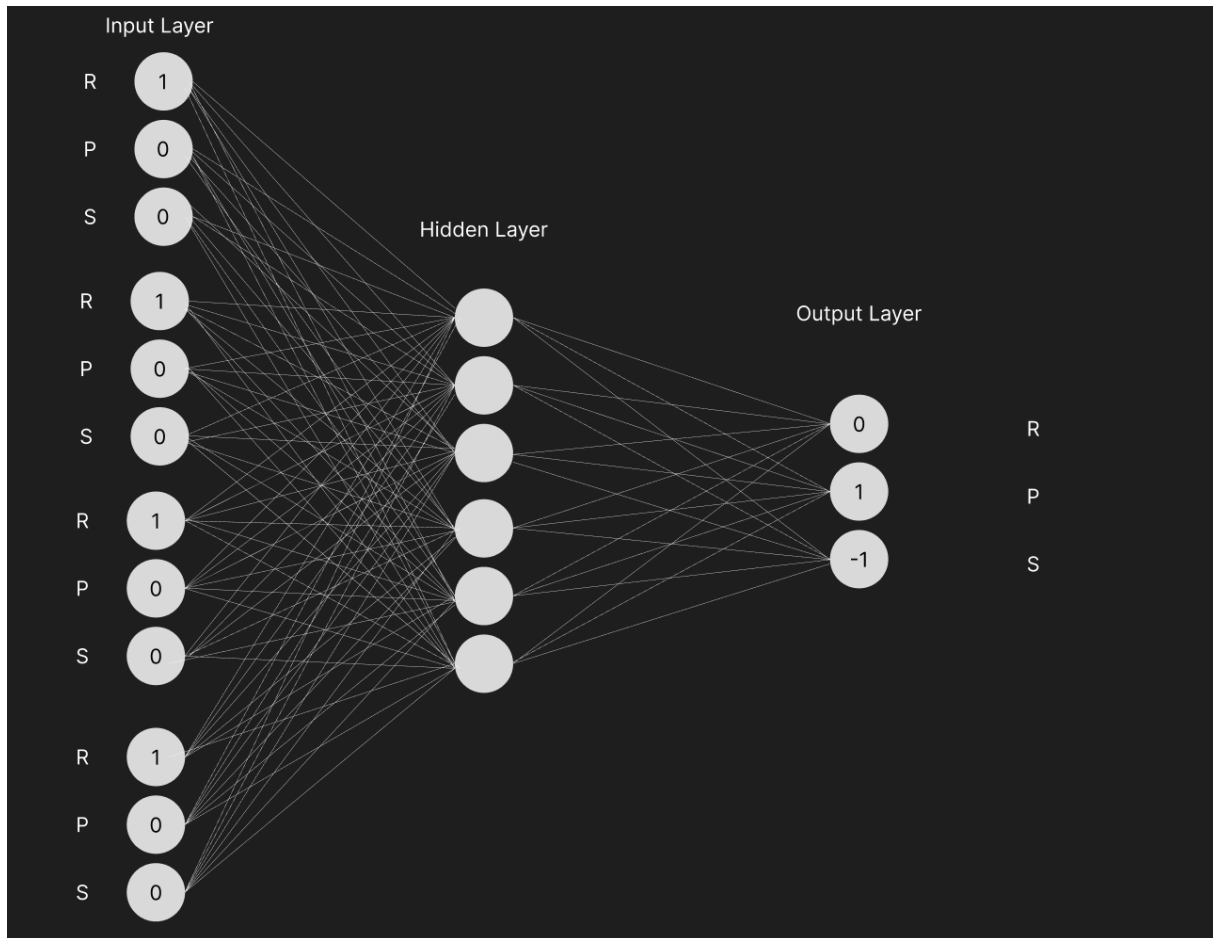


Figure 2: Caption

chains, subsequently being changed and contributing to the outputs of these chains. Forward propagation is the method used for passing input data, and receiving solutions corresponding to it, and this is a very important aspect when it comes to the learning of a ANN, which is used by backward-propagation to subsequently improve the ANN. To propagate through the network, each node uses its input, weight from that node and its bias to calculate its own weighted input. This weighted input is then fed through an activation function and used as output to the next node (Or just as output if you're on the output layer). Z_1 and A_1 represent the weight and activation function respectively:

$$Z_1 = XW_0 + b_0 \quad (1)$$

$$A_1 = f(Z_1) \quad (2)$$

This activation can then be used as input to the next node to calculate its' weighted input.

$$Z_2 = A_1W_1 + b_1 \quad (3)$$

2) *Backpropagation:* Considered as one of the most important aspects of an ANN, backward-propagation sets the tone for the fundamental learning of a ANN, and allows an ANN to generalize solutions over a problem space corresponding to very high amounts of problems that need to be solved. Backward-propagation relies on the use of a cost function which calculates the error between the predicted output and the actual output called the "cost"

or "loss", and based on this value adjusts the weights of the neural network proportionally in order to improve the costs at later stages using the same input data. As it is desired for the predicted output and real output to be very similar, it is then natural for the cost to also be as small as possible, thus the responsibility of the back-propagation is to adjust the internal weights of nodes in order to minimise this.

The main method that back-propagation relies on is the process of gradient descent which is responsible for adjusting the value of a weight depending on how dependent the output is on that particular weight. A method of looking at gradient descent is to imagine a 3D plane with contours and hills, and when a ball is dropped from any point, the ball will roll to the lowest point. This point is known as a local minimum. In the context of a ANN, the plane consists of "N" dimensions corresponding to "N" inputs, and the ball corresponds to the output eventually reaching it's optimum or minimum value in said plane.

F. Hyperparameters

Hyperparameters are parameters or "settings" which tells the agent what exactly it should do, when it should stop, how much data it's getting etc.

It is crucial to find a balance for these parameters so that the neural network can learn as efficiently as possible.

1) *The learn-rate:* One hyperparameter that plays an important role in back-propagation, is the learning rate, which is a scalar value that determines how much each of the weights change through each iteration of back-propagation. But when explored in more detail, the higher the learning rate does not always mean a more efficient ANN. When making the learning rate too large, the neural network tries to learn too quickly (ruins effectiveness), which might cause it to overshoot a lot of it's values, since the weights and biases hold a delicate balance. This causes it to not make any kind of consistent progress. On the flip side, making the learning rate too small causes the convergence of a solution to take too much time, thus ruining efficiency. We need to find some sort of balance between a high and low learn rate for the specific problem. Then the neural network will learn efficiently and effectively.

2) *Choosing an effective activation function:* Since the activation function is crucial to solving the rock paper scissors problem effectively, some effort needs to be made to choose the type of activation function. In the first attempt of solving the problem, a sigmoid activation function was used, however this proved to be inefficient in combination with the one hot encoding method prescribed. (As mentioned in "The activation function" section, the network does not see a difference between 2 moves due to one hot encoding. An example of an output is [0, 1, 0]. This defines a case where paper is the best move to play, which would mean that the neural network is expecting the opponent to play rock. With this in mind, the worst case scenario for a decision would be scissors. A more effective way of presenting the output would thus be [0, 1, -1], where the network doesn't really mind playing rock, it wants to play paper and it wants to avoid playing scissors. This type of output can be achieved when the activation function is tanh.

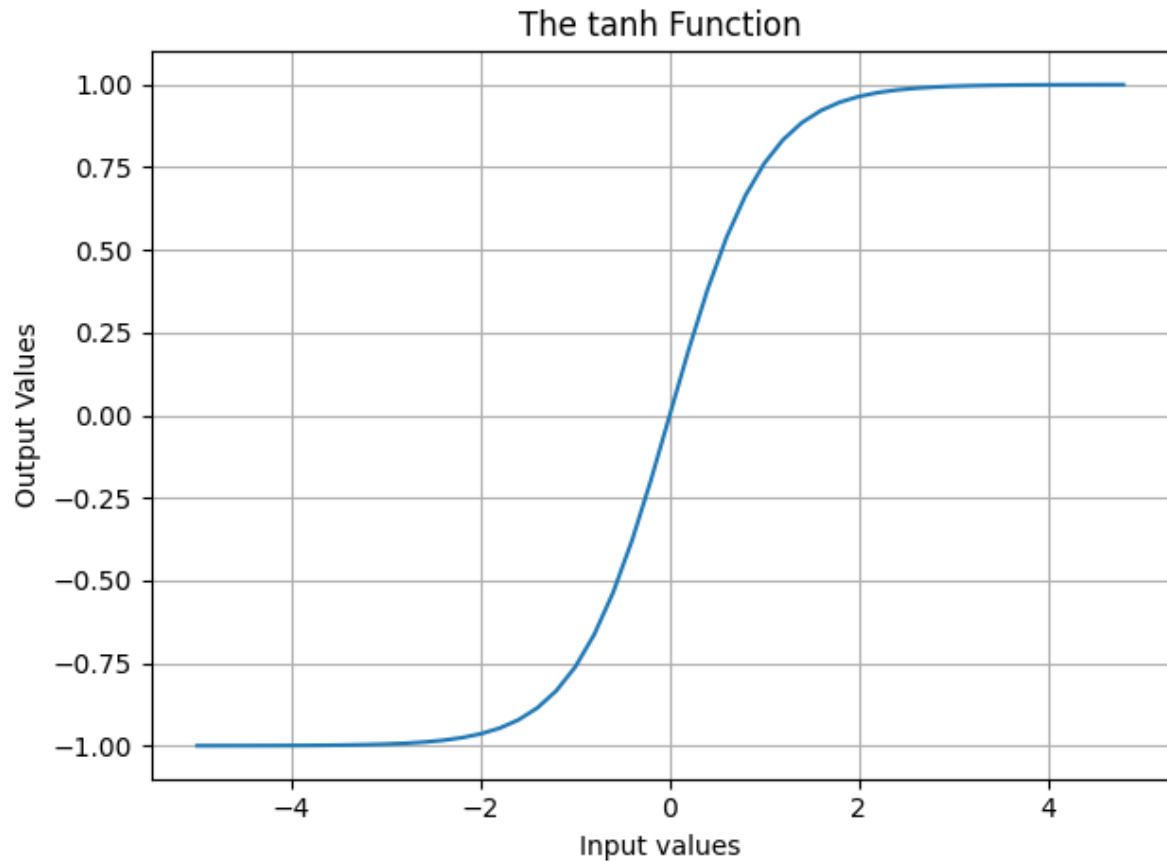


Figure 3: The TanH function

3) *Stopping criteria:* There are numerous ways for the neural network to decide when to stop. This parameter is known as the error convergence criterion. A common approach to this is to calculate the absolute value of the difference between the current error and the previous error. The algorithm should terminate when this reaches a specified threshold.

III. METHOD

Creating the neural network is broken into 3 different steps

- 1) Initialisation
- 2) Training
- 3) Testing

A. Initialisation

The neural network will be created using 2 classes, one of which is the neural network class itself, and the second one being the layer class, which will hold all the relevant information regarding nodes, weights and biases. The neural network will be initialised by defining the amount of input, hidden, and output nodes the system will have. After this, each layer will be set up by initialising random weights between the value of -1 and 1. The random initialisation can be more versatile, but for the sake of simplicity, initialising it between -1 and 1 would suffice. Each bias starts out with a value of 0.

B. Training

The 2 methods that will be used in training is forward propagation and backpropagation.

Backpropagation relies on finding the gradient that each weight and biases should move in to lower the effective cost. Lowering the cost means that you are closer to the optimal solution. These are the steps that the neural network is going to follow in order to learn:

- 1) Calculate the output of a single data point propagating forward through the neural network. When forward propagating, all the important parameters of each layer will be stored like the weighted inputs and activation value of each node. The output of the single data point will be measured with regards to the expected output by using the cost function.
- 2) After forward propagation is complete, the neural network will calculate in which direction each weight should move with backpropagation, starting at the output layer and moving backwards to calculate each value. (This will be discussed further below)
- 3) After calculating this value for each and every weight and bias, the process will be repeated for the entire batch of training data, storing and saving the direction each weight and bias should go in.
- 4) After the network has went through all of the data samples, an average direction for each weight of a node can be calculated and applied. When this update is applied, the neural network has "learned".

The goal is to just find out in which direction the weight should go in to lower the cost and move it in that direction (proportionally to the learning rate)

Using the chain rule and equation 1, it possible to see how a change in weight causes a change in cost. The change in weight causes a change in weighted input (equation 1), this change in weighted input causes a change in activation value (equation 2), which causes a change in output (equation 3) and therefore affects the cost function.

$$\frac{\partial J}{\partial W_2} = \frac{\partial Z_3}{\partial W_2} \cdot \frac{\partial \hat{Y}}{\partial Z_3} \cdot \frac{\partial J}{\partial \hat{Y}} \quad (4)$$

Calculating $\frac{\partial z}{\partial W}$ is quite simplistic when looking at equation 1. Thus, $\frac{\partial z}{\partial W} = X$ (or the activation/input value of that node).

$$\frac{\partial Z_3}{\partial W_2} = A_2 \quad (5)$$

Calculating $\frac{\partial \hat{Y}}{\partial Z_3}$ is found to be the derivative of the activation function.

$$\frac{\partial \hat{Y}}{\partial Z_3} = f'(Z_3) \quad (6)$$

Similarly, calculating $\frac{\partial J}{\partial \hat{Y}}$ can be found by looking at the cost/loss function:

$$J = \frac{1}{2} \sum_v (Y - \hat{Y})^2 \quad (7)$$

In equation 7, t_k and z_k refers to the target and return values of a single output node k . The rest of the math is fairly intuitive:

$$\frac{\partial J}{\partial \hat{Y}} = -(Y - \hat{Y}) \quad (8)$$

If we let δ_3 be equal to equation 5 multiplied with equation 8, we get the following:

$$\delta_3 = \frac{\partial \hat{Y}}{\partial Z_3} \cdot \frac{\partial J}{\partial \hat{Y}} \quad (9)$$

$$\delta_3 = -f'(Z_3) \cdot (Y - \hat{Y}) \quad (10)$$

This leads to the weights being updated in to a value where the final cost is lower:

$$W_2 = W_2 - \eta \frac{\partial J}{\partial W_2} \quad (11)$$

$$W_2 = W_2 - \eta \frac{\partial Z_3}{\partial W_2} \cdot \frac{\partial \hat{Y}}{\partial Z_3} \cdot \frac{\partial J}{\partial \hat{Y}} \quad (12)$$

$$W_2 = W_2 - \eta A_2 \cdot \delta_3 \quad (13)$$

Using the equations mentioned above, along with object oriented programming, the new weights can be calculated very efficiently and can be used to make the neural network learn.

IV. RESULTS

The fully implemented neural network gave the following results:

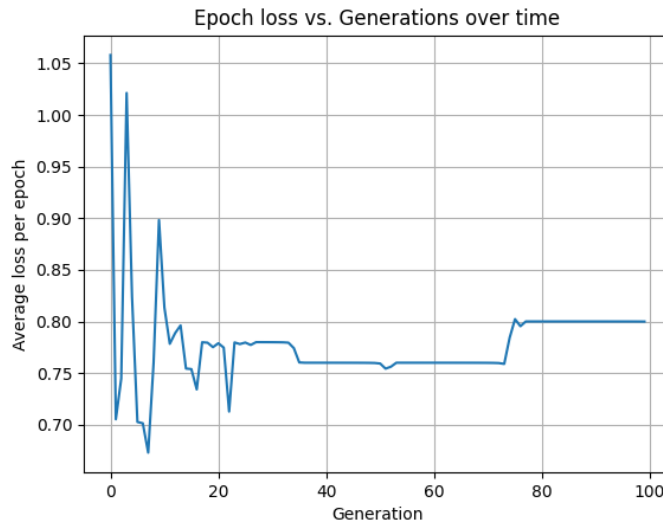


Figure 4: Neural Network struggling to learn

Looking at the epoch loss of this network, it is very jagged and unpredictable. It starts off with a major decline in L_2 losses. The lowest that the L_2 loss goes is to around 0.7, which is high for this neural network. The local minimum also seems to be unstable and the neural network goes back to a higher L_2 loss in the following generation. The L_2 loss also seems to plateau around 0.76, before jumping back up to a higher average L_2 loss. This is due to the learn-rate being 0.8

Experimenting with a low learn rate leads to the following:

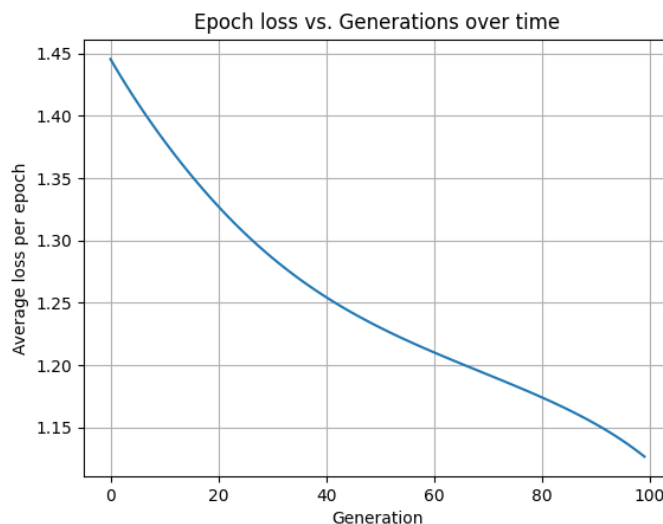


Figure 5: Neural Network learning slowly

In this figure, the loss per epoch is decreasing at a steady rate, not increasing at all. It is not very fast though, which is the payoff to make for accuracy. The lowest loss it gets is 1.15,

which is more than a 200% increase from the previous figure's lowest value. Given more time, it is likely to have performed better. The learn-rate in this set up was 0.005

When choosing a value between 0.005 and 0.8, a much more balanced solution was found:

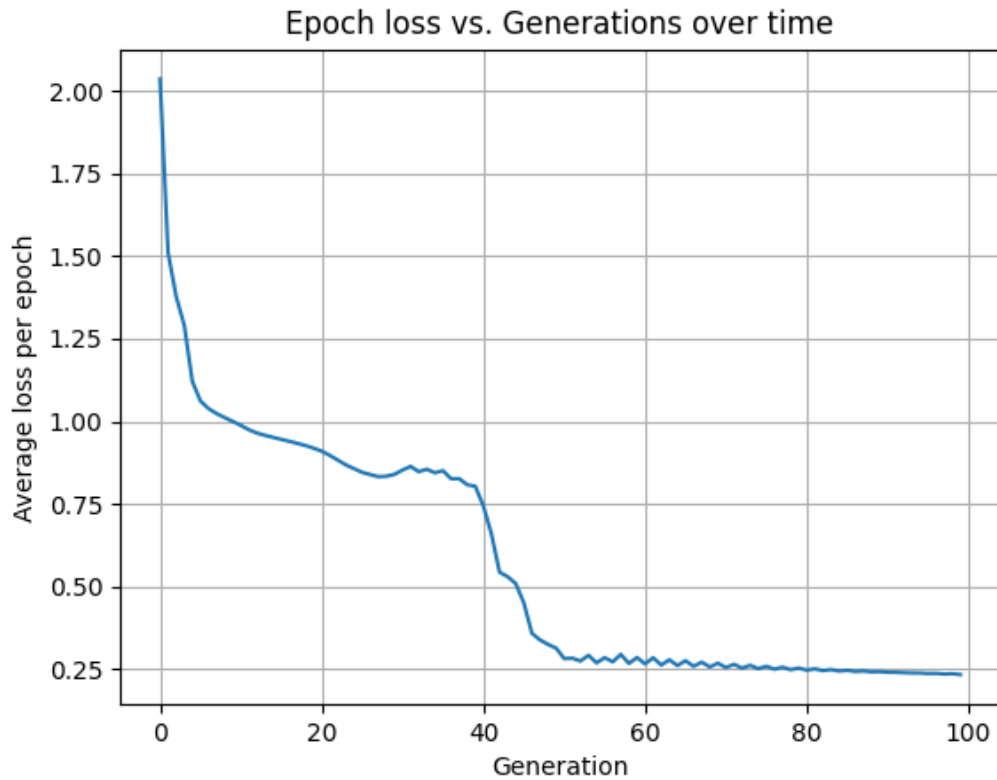


Figure 6: Neural Network learning fast and effectively

This run started with a very high L_2 loss per epoch, but exponentially decreased to a lower value initially. Over time, the value seemed to plateau around 0.24, which is relatively good compared to the other results.

When attempting to play against simple agents, the following outputs were observed:

```
total run time: 471.24 seconds

only_rock.py: won 0.0% of matches (0 of 100)
  won 0.0% of rounds (53 of 1000000)
  avg score -9947.2, net score -994725.0

mainAgent.py: won 100.0% of matches (100 of 100)
  won 99.5% of rounds (994778 of 1000000)
  avg score 9947.2, net score 994725.0
```

Figure 7: Agent playing against only_rock.py

```
total run time: 133.61 seconds

only_paper.py: won 0.0% of matches (0 of 100)
  won 0.0% of rounds (51 of 1000000)
  avg score -9498.2, net score -949821.0

task3Attempt2.py: won 100.0% of matches (100 of 100)
  won 95.0% of rounds (949872 of 1000000)
  avg score 9498.2, net score 949821.0
```

Figure 8: Agent playing against only_paper.py

```
total run time: 476.14 seconds

only_scissors.py: won 0.0% of matches (0 of 100)
  won 0.0% of rounds (68 of 1000000)
  avg score -9998.1, net score -999807.0

mainAgent.py: won 100.0% of matches (100 of 100)
  won 100.0% of rounds (999875 of 1000000)
  avg score 9998.1, net score 999807.0
```

Figure 9: Agent playing against only_scissors.py

These were the the final results taken for task 3. Task 2 performed similarly, just with a lower time complexity since it doesn't have to learn every "n" moves. It still won 100% of matches, just as task 3, which is why task 2's results isn't included in the report.

The average time it took the neural network for task 3 to learn and play against these bots were 479.6 seconds, which is around 7.98 minutes. The agent does win 100% of matches and almost 100% of rounds. By winning against these simplistic agents, the agent is shown to perform well when learning as it's playing, or when set up with a predefined list of weights and biases.

Against more difficult bots, the match win-rate is still the same, with the only difference being the rounds win percentages lowering.

```
total run time: 149.29 seconds

beat_common.py: won 0.0% of matches (0 of 100)
  won 21.9% of rounds (218702 of 1000000)
  avg score -2962.1, net score -296214.0

task3Attempt2.py: won 100.0% of matches (100 of 100)
  won 51.5% of rounds (514916 of 1000000)
  avg score 2962.1, net score 296214.0
```

Figure 10: Agent playing against beatCommon.py

```
total run time: 148.40 seconds

beat_previous.py: won 0.0% of matches (0 of 100)
  won 7.0% of rounds (70079 of 1000000)
  avg score -4540.3, net score -454029.0

task3Attempt2.py: won 100.0% of matches (100 of 100)
  won 52.4% of rounds (524108 of 1000000)
  avg score 4540.3, net score 454029.0
```

Figure 11: Agent playing against beatPrevious.py

It is also noticeable how the time has decreased significantly from the simple bots, this is due to a decrease in a change in "n". In the simple bots, the bots learned after every 1 move, where with the more complex bots, the bot only learned after every 10 moves for simplicity

V. DISCUSSION

As can be seen in the results, the ANN was successfully configured in order to minimize the overall error through using forward propagation in order to determine the best move in a RPS runner. This therefore means that the ANN can effectively determine the correct result over a number of epochs when trained off of training data, used to mirror real life playing. But this success is dependant on the correct initialisation of hyperparameters that ultimately determine how effectively the ANN can reach the end goal, and how much time it takes.

A. *How the learn-rate affects the neural network*

The hyperparameter that had a tremendous affect on the success of the ANN, was the alterations done to the learning rate of the ANN. The learning rate determines the speed at which the ANN is able to learn by adjusting certain parameters such as the weights between adjacent nodes. It might be intuitive to think that the higher the learning rate, the more effective the ANN would perform, but through experimentation this is seen as not true, as when the learning rate is initialised as too high, the ANN performs in a sort of erratic nature never ending up at the optimal solution, but rather just "bouncing" around it. As seen in Figure 4, the ANN tries to overcompensate for its errors, thus never being able to find the true solution, and even over time becoming worse as the error might increase as a result of this high learning rate. Thus the use of a high learning rate is not optimal in finding the solution in a faster manner.

But we run into another problem concerned with the learning rate, and this is when the initialisation of said learning rate is too small, causing a negative affect in the ANN's time complexity. As seen in figure 5, the ANN does effectively learn this time, but is no means efficient in its learning. Although a small learning rate increases the chances of honing in on the optimal solution when compared to the very large learning rate, this comes with a disadvantage as the ANN then takes too much time, as only very small movements can be made towards the optimal solution in the problem space. As seen in Equation ??, a small learning rate will cause a small change in the recalculation of W_2 , as well as W_1 which is represented by the same formula. This small change in weight limits the speed at which the ANN can learn, thus decreasing the overall efficiency of the neural network.

A common method to learn more effectively is to have a large learn-rate initially, but decrease the learn-rate over time. This is a very useful approach, but in the end, finding a good constant learn-rate solved the problem, as can be seen in figure 6. Of course, this solution is not guaranteed to be the optimal solution, known as the global minimum, but it is a good start.

It is thus important as seen in Figure 6 to initialise a learn rate which is big enough in order to efficiently makes changes to the ANN in order to reach the optimal solution quickly, but small enough so that the ANN does not act in an erratic nature, as is able to effectively hone in on the optimal solution. Therefore the correct implementation and experimentation of the learning function was very important in establishing an effective and efficient working ANN.

B. *How the hidden layer affects the neural network*

There are 2 main methods to increase complexity for the neural network.

- 1) Increase the number of nodes in the hidden layer.
- 2) Increase the amount of hidden-to-hidden layers.

When increasing the number of nodes in the hidden layer, there is more space for tweaking and more precise tuning to the perfect values. This is very beneficial, but can quickly be detrimental to the speed of the neural network, increasing the time and space complexity of the network. Also, after a certain number of hidden layer nodes, the complexity of the problem space might already be encapsulated within the network, so adding more nodes will be redundant and just waste time and space.

Adding more hidden-to-hidden layers to the network increases the depth of the network. Deeper networks (networks with more hidden layers) can learn more complex representations of data than shallower networks. However, adding too many hidden layers can lead to overfitting, where it essentially "memorises" the answers to problems it has seen, but performs poorly against unseen data.

C. How the loss is affected while training the data

When the neural network is set up with appropriate parameters, the loss will initially be high. Initially, the loss will decrease very rapidly, as the neural network is learning common patterns. As it learns, the loss will still decrease, but at a steadier pace. This is due to complex patterns taking more time to learn, and the solution found is not guaranteed to be the optimal solution.

D. Effect of online learning on the performance of the agent

A factor to look at is the performance of the agent when exposed to online learning, as when compared to only training data, in order to evaluate the effectiveness of the agent as a whole. Weights were taken from Task 2 in order to start with a confident assumption of nodal weights, as well as to increase the efficiency of the agent in the beginning rather than learning from scratch each time, just giving a boost in overall effectiveness over a shorter amount of time. This meant that the agent could work off previous knowledge of generalised RPS games, and further be able to specialize according to the opponent it was playing, and specialize it did. It can be seen in Figure 7, 10 and 9 that the agent won 100% against its respective opponents, confirming the effectiveness of the ANN and confirming its ability to learn and win over multiple epochs. Being able to only learn after "n" moves, the ANN still showed to be very efficient as it met requirements from the brief stating that "this agent can play 100 matches of 10 000 rounds each against a simple bot (like only_rock.py or only_scissors.py) in less than 10 minutes when running on a single core", confirming its status of being a fast and efficient ANN.

VI. CONCLUSION

The purpose of this practical was to research and implement a working Artificial Neural Network, or ANN for short, in order to effectively and efficiently learn and adapt to win games in the Rock-Paper-Scissors format. Through research and implementation of ANN concepts such as forward- and backpropagation algorithms, hyperparameters such as nodal size, growth rate and activation function algorithms and lastly training and stopping conditions, we were able to create an ANN that could effectively win opponents through training data as well as independent online games, as well as do this in an efficient manner showing the usefulness of ANNs as a whole in the world of artificial intelligence and artificial learning networks. This further proves the purpose of the artificial intelligence in the real world as a method of problem-solving that can be fully automated in order to learn and perform accurately and effectively on its own without the need for human input. ANNs give us the ability to perform and complete tasks that range from very simple, to tasks that can reach a complexity unfathomable to the mind, thus proving the usefulness it has to the ever evolving and ever growing world of automation.

VII. CODE

```
1 import math
2 import numpy as np
3 import random
4
5 #u22523538: Dylan Schmidt
6 #u22498916: Crouse Louw
7
8
9 #Converts an input to usable nodes for the agent
10 def CalculateInputActivation(inputElement):
11     #Inputs gets converted as follows:
12     #R = [1 0 0]
13     #P = [0 1 0]
14     #S = [0 0 1]
15     if (inputElement == 'R'):
16         return [1,0,0]
17     elif (inputElement == 'P'):
18         return [0,1,0]
19     else:
20         return [0,0,1]
21
22
23
24
25 #Initialising settings
26 #Settings
27 numInputFeatures = 12
28 numOutputNeurons = 3
29 numNeuronsHiddenLayer = 40
30 learningRate = 0.2
31 numberOfEpochsToRunFor = 1
32 numhiddenLayer = 1
33
34 C = numOutputNeurons
35 D = numInputFeatures
36 H = numNeuronsHiddenLayer
37
38 #Loss function
39 def CalculateLoss(target , result):
40     sum = 0
41     for i in range (0,len(target)):
42         sum += math.pow((target[i] - result[i]), 2)
43     return sum/2
44
45 #Tanh is used as activation function
46 def ActivationFunction(weightedInput):
47     return math.tanh(weightedInput)
48
49 #Derivative , used for backpropogating
50 def ActivationDerivative(weightedInput):
51     return (1 - (math.tanh(weightedInput))**2)
52
53 #returns a random value between 0 and 1
54 def RandomizeInitial():
55     return random.random()
```

```

56
57
58
59 # -----NEURAL NETWORK-----
60 #Inputs:
61 #           R   -   1
62 # P1M1      P   -   0           = R
63 #           S   -   0
64
65 #           R   -   0
66 # P2M1      P   -   0           = S
67 #           S   -   1
68
69 #           R   -   0
70 # P1M2      P   -   1           = P
71 #           S   -   0
72
73 #           R   -   1
74 # P2M2      P   -   0           = R
75 #           S   -   0
76 #Outputs:
77
78 #           R   -   1
79 #           P   -   0           = R   (Move that the neural network
80 #           S   -   0           should play)
81
82
83 #Function to return the winning move against another move
84 def WinMove(move):
85     if move == 'R':
86         return 'P'
87     elif move == 'P':
88         return 'S'
89     return 'R'
90
91 #This puts the input into a usable format
92 class data:
93     def __init__(self, matchHistory, inputResult): #Matchhistory ex =
94         [R,R,R,P], inputresult ex = 'R'
95         array = []
96         for i in range(0,4):
97             array = array + CalculateInputActivation(matchHistory[i])
98             self.inputs = array #Input is in the form of 12 numbers 0
99             through to 1
100             self.expected = CalculateInputActivation(WinMove(inputResult)
101             )
102             if (self.expected[0] == 1):
103                 self.expected[1] = -1
104             elif (self.expected[1] == 1):
105                 self.expected[2] = -1
106             else:
107                 self.expected[0] = -1
108 #The data class has 2 values, namely inputs and expected. Inputs is
109 #the match history, outputs is the expected output
110 #An example of these variables stored are

```

```

107
108 #Inputs = [1 0 0 1 0 0 1 0 0 1 0 0] #Explanation: [Rock Rock Rock
      Rock]
109 #Outputs = [0 1 -1]# Explanation: [Rock is okay,      Paper is perfect ,
      Scissors is worst case]
110
111
112 class Layer:
113     def __init__(self , numIn , numOut):
114         self.numInputs = numIn
115         self.numOutputs = numOut
116         self.weights = np.zeros([numIn , numOut])
117         self.biases = np.zeros(numOut)
118         self.gradientB = np.zeros(numOut)
119         self.gradientW = np.zeros([numIn , numOut])
120         self.inputs = np.zeros(numIn)
121         self.weightedInputs = np.zeros(numOut)
122         self.activations = np.zeros(numOut)
123
124         self.initializeWeights()
125
126     def initializeWeights(self):
127         for i in range(0, self.numInputs):
128             for j in range(0, self.numOutputs):
129                 #Initialise weights to a random value between -1 and
                    1
130                 self.weights[i][j] = ((RandomizeInitial()*2)-1)
131
132
133     def LayerOutputs(self , inputs):
134         #This calculates the confidence
135         #output = activation(w*input + b)
136         self.inputs = inputs
137
138         neuronActivations = np.zeros(self.numOutputs)
139         for numOut in range(0, self.numOutputs):
140             weightedInput = self.biases[numOut]
141             for numIn in range(0, self.numInputs):
142                 weightedInput += inputs[numIn]*self.weights[numIn][
                    numOut]
143             #Saving these values for backpropogation
144             self.weightedInputs[numOut] = weightedInput
145             neuronActivations[numOut] = ActivationFunction(
                    weightedInput)
146             self.activations[numOut] = neuronActivations[numOut]
147         #returns an array of output confidence
148         return neuronActivations
149
150     #This function applies all calculated gradients to their weights
        to lower cost
151     def ApplyChanges(self , learnrate):
152         for i in range(0, self.numOutputs):
153             self.biases[i] -= self.gradientB[i] * learnrate
154             for j in range(0, self.numInputs):
155                 self.weights[j][i] -= self.gradientW[j][i] *
                    learnrate
156

```

```

157     #This function calculates the gradients for backpropagation
158     def calculateGradients(self, nodeValues):
159         for i in range(0, self.numOutputs):
160             for j in range(0, self.numInputs):
161                 #Cost derivative with respect to weight
162                 derivativecost = self.inputs[j]*nodeValues[i]
163                 self.gradientW[j][i] += derivativecost
164                 #Cost derivative with respect to biase
165                 derivativecostbiase = nodeValues[i]
166                 self.gradientB[i] += derivativecostbiase
167     #this function doesn't return anything, it is purely to calculate
        change in gradient
168
169     #The cost of a node/neuron
170     def NodeCost(self, target, expected):
171         return 0.5*math.pow((target - expected), 2)
172
173     #The derivative of the nodeCost
174     def NodeCostDerivative(self, outputActivation, expectedOutput):
175         return (outputActivation-expectedOutput)
176
177     # This function returns an array of nodevalues used to calculate
        the costs of neurons.
178     # These values are used to backpropagate
179     # This is used for hidden to output layer
180     def calculateNodeOut(self, expectedOut):
181         nodeValues = np.zeros(len(expectedOut))
182         for i in range(0,len(expectedOut)):
183             costDerivative = self.NodeCostDerivative(self.activations
                [i], expectedOut[i])
184             activationderivative = ActivationDerivative(self.
                weightedInputs[i])
185             nodeValues[i] = costDerivative * activationderivative
186         return nodeValues
187
188     # This function is the same as the previous one, only difference
        is that
189     # it is used for hidden to hidden layer
190     def calculateHiddenValues(self, oldLayer, oldNodes):
191         newNodes = np.zeros(self.numOutputs)
192         for i in range(0, len(newNodes)):
193             newNode = 0
194             for oldnode in range(0, len(oldNodes)):
195                 weightedDerivative = oldLayer.weights[i][oldnode]
196                 newNode += weightedDerivative * oldNodes[oldnode]
197             newNode *= ActivationDerivative(self.weightedInputs[i])
198             newNodes[i] = newNode
199         return newNodes
200
201     #This function resets all weights and biases in a layer
202     def clearWeightsAndBiases(self):
203         self.weights = np.zeros([self.numInputs, self.numOutputs])
204         self.biases = np.zeros(self.numOutputs)
205     #This will be used to initialise a layer with weights and biases
206     #it essentially loads up a previous save for a given layer
207     def AssignGivenWeights(self, inputWeightData):
208         #Setting weights

```

```

209         for i in range(0, self.numInputs):
210             for j in range(0, self.numOutputs):
211                 self.weights[i][j] = inputWeightData[i][j]
212         #Setting biases
213         for k in range(0, self.numOutputs):
214             self.biases[k] = inputWeightData[self.numInputs][k]
215
216         #This does the inverse of the function above, it retrieves
         weights and biases for a given layer
217     def getWeightsAndBiases(self):
218         Result = []
219         biasesarray = []
220         weightsArray = []
221         for i in range(0, self.numInputs):
222             weightsArray = []
223             for j in range(0, self.numOutputs):
224                 weightsArray.append(self.weights[i][j])
225             Result.append(weightsArray)
226         for k in range(0, self.numOutputs):
227             biasesarray.append(self.biases[k])
228         #The array stores the weights first, as arrays of arrays,
         then adds the biase in the bottom
229         Result.append(biasesarray)
230         return Result
231
232     #This class is the neural networkk, where layers and everything gets
         initialised
233     class NeuralNetwork:
234         def __init__(self, layerNodes):
235             #Initialise array of layers for the neural network
236             self.layers = []
237             #layerNodes is an array that contains all the input, hidden,
         and output data
238             #If I want to create an array with 2 hidden layers with 20
         nodes per hidden layer,
239             #layerNodes will look as follows = [12, 20, 20, 3]
240             #Where 12 is the input nodes and 3 is the output nodes
241             #Creating layers
242             for i in range(0, len(layerNodes)-1):
243                 self.layers.append(Layer(layerNodes[i], layerNodes[i+1]))
244
245             #This function takes all the inputs and feeds it through
         layer by layer
246         def CalculateNetworkOutput(self, inputs):
247             OutputToInput = inputs
248             for layer in self.layers:
249                 #Each layer's output is the next layers input
250
251                 OutputToInput = layer.LayerOutputs(OutputToInput)
252             #Return the output of the final layer (output layer)
253             return OutputToInput
254
255         #This function calculates the difference between the expected and
         output value,
256         #thus calculating the cost/loss
257         def Loss(self, datapoint):
258             output = self.CalculateNetworkOutput(datapoint.inputs)

```

```

259     #The loss is only calculated in the output layer
260     outputlayer = self.layers[len(self.layers)-1]
261     cost = 0
262     for i in range(0, len(output)):
263         cost += outputlayer.NodeCost(output[i], datapoint.
            expected[i])
264     #The cost is the summation of all the "errors" in the neural
        network
265     return cost
266
267
268     #This function calls each layer to calculate and update their
        gradients
269     #This is the heart of the learning of the neural network
270     def UpdateAllGradients(self, singleDatapoint):
271         #Forward propogation, saving necessary values
272         self.CalculateNetworkOutput(singleDatapoint.inputs)
273
274         outputLayer = self.layers[len(self.layers)-1]
275         nodeValues = outputLayer.calculateNodeOut(singleDatapoint.
            expected) #output nodes
276         outputLayer.calculateGradients(nodeValues)
277         #calculate for every hidden layer
278         for hiddenIndex in range(len(self.layers) - 2, -1, -1):
279             hiddenLayer = self.layers[hiddenIndex]
280             nodeValues = hiddenLayer.calculateHiddenValues(self.
                layers[hiddenIndex+1], nodeValues)
281             hiddenLayer.calculateGradients(nodeValues)
282         #function doesn't return anything as it only updates values
283         return
284
285     #This function takes everything and puts it together to create
        the backpropagation function
286     def LearnBackPropagate(self, trainingBatch, learnrate):
287         #Trains for every batch
288         for data in trainingBatch:
289             self.UpdateAllGradients(data)
290
291         #The change only gets applied AFTER training, by the average
            of the changes
292         #(proportionally to the learnrate)
293         self.ApplyGradientsToLayers(learnrate/len(trainingBatch))
294         self.ClearGradients()
295         #returns the epoch loss of the layer
296         return
297
298     def LearnBackPropagateWithLoss(self, trainingBatch, learnrate):
299         #Trains for every batch
300         for data in trainingBatch:
301             self.UpdateAllGradients(data)
302
303         #The change only gets applied AFTER training, by the average
            of the changes
304         #(proportionally to the learnrate)
305         self.ApplyGradientsToLayers(learnrate/len(trainingBatch))
306         self.ClearGradients()
307         #returns the epoch loss of the layer

```

```

308         return self.AverageLoss(trainingBatch)
309
310
311     #Clears the gradients in all layers (NOT the weights and biases ,
312     #just the gradients they should change for)
313     def ClearGradients(self):
314         for layer in self.layers:
315             layer.gradientB = np.zeros(layer.numOutputs)
316             layer.gradientW = np.zeros([layer.numInputs, layer.
317                                     numOutputs])
318
319         return
320
321     #Calculates the average loss of an epoch
322     def AverageLoss(self, GroupOfData):
323         totalCost = 0
324         for datapoint in GroupOfData:
325             cost = self.Loss(datapoint)
326             totalCost += cost
327         return totalCost/len(GroupOfData)
328
329     #This manually calculates the gradient to update. DO NOT USE for
330     #production
331     #Uses normal calculus, no derivatives
332     def ForwardPropogation(self, Data, learningrate):
333         h = 0.00001 #Make some small change to weight, see howw that
334         #affects output
335         #Find original Loss of network
336         originalLoss = self.AverageLoss(Data)
337         for EachLayer in self.layers:
338             for eachInput in range(0, EachLayer.numInputs):
339                 for eachOutput in range(0, EachLayer.numOutputs):
340                     #Change weight by h, find difference in network,
341                     #get gradient
342                     EachLayer.weights[eachInput][eachOutput] += h
343                     newLossOfNetwork = self.AverageLoss(Data)
344                     EachLayer.weights[eachInput][eachOutput] -= h #do
345                     #not actually change weight, just observe
346                     #changes
347                     difference = newLossOfNetwork - originalLoss
348                     gradient = difference/h
349                     #print(EachLayer.gradientW[eachInput][eachOutput
350                     ])
351                     #print(gradient)
352                     EachLayer.gradientW[eachInput][eachOutput] =
353                     gradient
354
355                 for k in range(0, EachLayer.numOutputs):
356                     EachLayer.biases[k] += h
357                     newLossOfNetwork = self.AverageLoss(Data)
358                     EachLayer.biases[k] -= h
359                     difference = newLossOfNetwork - originalLoss
360                     gradient = difference/h
361                     EachLayer.gradientB[k] = gradient
362
363         #Update values
364         self.ApplyGradientsToLayers(learningrate)
365

```

```

356     #This function returns the move that the neural network should
        get
357     def getWin(self , inputs):
358         results = self.CalculateNetworkOutput(inputs)
359         max_val = results[0]
360         max_idx = 0
361         #Finds the highest confidence value
362         for i in range(1, len(results)):
363             if results[i] > max_val:
364                 max_val = results[i]
365                 max_idx = i
366         #output = [a b c] where a = rock , b = paper , c = scissors
367         if max_idx == 0:
368             return 'R'
369         elif max_idx == 1:
370             return 'P'
371         else:
372             return 'S'
373
374     #Goes through the layers , telling each layers to change weights
        and biases by
375     #looking at the gradients
376     def ApplyGradientsToLayers(self , learnrate):
377         for layer in self.layers:
378             layer.ApplyChanges(learnrate)
379
380     #Goes through the layers , telling each layers to reset weights
        and biases gradients
381     def clearWeightsPerLayer(self):
382         for layer in self.layers:
383             layer.clearWeightsAndBiases()
384     #Go through each layer collecting weights and biases ,
385     #this value is stored somewhere to save your neural network
        progress
386     def getWeightsPerLayer(self):
387         Result = []
388         for layer in self.layers:
389             Result.append(layer.getWeightsAndBiases())
390         return Result
391
392     #The opposite of the function above
393     #Loads a saved neurel network
394     def SetWeightsPerLayer(self , layerData):
395         for i in range(0, len(layerData)):
396             self.layers[i].AssignGivenWeights(layerData[i])
397
398
399     #WeightsInpToHidden:
400     #1      [0.5 , 0.2]
401     #2      [0.1 , 0]
402     #3      []
403     #4      []
404     #5      []
405     #6      []
406     #7      []
407     #8      []

```

2

```

408 #9      []
409 #10     []
410 #11     []
411 #12     []
412 #13     Biases=[1, 0]
413
414
415 def backpropogation(inputData , predictedValues , WeightsInpToHidden ,
    WeightsHiddenTohidden , WeightsHiddentoOutput , learnrate ,
    numEpochs):
416     resultInpToHid = []
417     resultHidToHid = []
418     resultHidToOut = []
419     resultsLossPerEpoch = []
420     #Create Neural network with specified sizes
421     #Building paramaters for initialising neural network
422     inputParamaters = []
423     inputParamaters.append(len(WeightsInpToHidden)-1)
424     for i in range(0,len(WeightsHiddenTohidden)+1):
425         inputParamaters.append(H)
426     inputParamaters.append(len(WeightsHiddentoOutput[0]))
427     #print("Neural Network created")
428     myNeural = NeuralNetwork(inputParamaters)
429     #Sets the weights to previously known values for neural network
430     weights = []
431     weights.append(WeightsInpToHidden)
432     if len(WeightsHiddenTohidden) != 0:
433         weights.append(WeightsHiddenTohidden)
434     weights.append(WeightsHiddentoOutput)
435     #Sets the weights per layer
436     #example: weights = [[layer1 weightsAndBiases], [layer2
        weightsAndBiases], [layer 3 weightsAndBiases]]
437     myNeural.SetWeightsPerLayer(weights)
438     learningData = []
439
440     #Puts the data into the correct format
441     for i in range(0,len(inputData)):
442         learningData.append(data(inputData[i] , predictedValues[i]))
443
444     #Does backpropogation in the neural network
445     for i in range(0 , numEpochs):
446         #print(i)
447         resultsLossPerEpoch.append(myNeural.LearnBackPropogate(
            learningData , learnrate))
448     # while ( currentCondition > numEpochs):
449     #     myNeural , loss = TrainWithGivenData(myNeural,2,learningData
        , learnrate)
450     #     resultsLossPerEpoch = resultsLossPerEpoch + loss
451     #     currentCondition = resultsLossPerEpoch[len(
        resultsLossPerEpoch)-2] - resultsLossPerEpoch[len(
        resultsLossPerEpoch)-1]
452
453     #Saves the weights from the neural network
454     results = myNeural.getWeightsPerLayer()
455     #print("Just collected all of the data")
456     resultInpToHid = results[0]
457     resultHidToOut = results[len(results)-1]

```

```

458     for i in range(1, len(results)-1):
459         resultHidToHid.append(results[i])
460     return [resultInpToHid, resultHidToHid, resultHidToOut,
            resultsLossPerEpoch]
461
462
463 InitialWeights = [[[0.336167721352624, -0.8186083367804304,
            0.0809513343521018, 0.3606828297506358, -0.3442843337728612,
            -0.33568777836377317, -0.3508019970949795, 0.336674871586882,
            0.5876493787602326, 0.8563683074109716, 0.9733661923852841,
            0.7923470552728532, 0.9426551761506212, 0.39364137655376047,
            0.8424552891393848, -0.5668148680585915, -0.3851040840008496,
            -0.35259723738282434, -0.5400108263706968, -0.13264175392327762,
            0.35064047620515, 0.8939264490964334, -0.2779299437406824,
            -0.11247279005553727, -0.010167426765101795, -0.868470020809911,
            0.28484687594093056, 0.12345250469351125, 0.47595511923005673,
            -0.3222289836100598, 0.8890771193131897, -0.47927691447592236,
            0.11523125583052155, 0.8564957003583354, -0.8087890145398846,
            0.37599641842111775, 0.34970067459853293, -0.28770403680191686,
            -0.16054960569069102, -0.3474064747103386], [0.461309282591249,
            1.02477154951481, -0.14393567711797503, 0.6473908480640184,
            -0.06163509872116947, -0.16896918579200848,
            -0.007242907363909366, 0.5909737513744464, -0.16490942292718627,
            -0.5898321141463317, -0.5684948012344376, -0.33839975525023785,
            -0.39159070349558334, -0.7809865439062607, -0.3160547737653059,
            0.11275690805631655, -0.6697293746582031, -0.7097944361004609,
            0.9477722715116084, -0.6544294094889967, -0.8963042576087693,
            -0.6428327070284129, 0.9695379536404538, 0.8345401468323392,
            0.6468260666531013, 0.771979951074747, 0.8048255045838886,
            0.46829059565756254, 0.1297141292752502, 0.35459757239423645,
            -0.08240264553053632, -0.7073128944568549, -0.7854152908431303,
            -0.7002813642994212, -0.15913746291995987, -0.8389083382054913,
            0.08116599971382638, 1.00461022215253, -0.30871382001329,
            0.1656960690765203], [0.44874950417046044, -0.38075244104423783,
            0.332771599693252, 0.11554336932491473, 0.23219746594817822,
464 -0.7881842659738642, 0.19734018004155743, -0.9137019769200262,
            0.14177987742638368, -0.6116399084938088, -0.08482618938464542,
            0.5933835911223212, -0.6617802714551353, -0.7245716055562362,
            -0.3937915842319024, 0.6474594263296755, 0.03539607082308757,
            -0.6070402573434434, 0.8273049172058863, 0.13209463868732538,
            -0.6268333742950639, -0.06681146959806333, 0.690048774694201,
            0.19658958091637505, -0.8519581353615213, 0.2441736130080034,
            -0.057695831906465364, 0.06677540589543401, -0.8923241110383531,
            -0.5567593794175195, 0.4656704100997905, -0.9205924432905347,
            -0.5361952114870568, 0.42618126374678145, -0.19338506511393352,
            -0.1724479554525013, 0.2156169457869321, -0.19036728420471039,
            0.7344156882973321, 0.160854928845583], [-0.7122507456791239,
            0.09969439479446034, -0.05746446620144725, -0.1318479038732956,
            -0.10609966819653, 0.995709349258153, -0.8181499029279088,
            -0.026394845940749555, -0.12078096804117142, 0.8563918416432602,
            0.8232288636087637, 0.15116042379835443, -0.7340928286491449,
            -0.8492002195075251, 0.2935713111396481, 0.8884013754426787,
            -0.6821454220090096, -0.8364914437687976, 0.9310232552896391,
            0.6786273122946109, 0.6139281693400404, -0.8948620184236894,
            0.048847995655941875, 0.37699032400608024, -0.6679452359545266,
            -0.45148102534679874, -0.8044330374925996, 0.7023225982697296,
            -0.457340103842369, -0.30017933310523987, 0.16429912166650792,

```

-0.5977546215652165, -0.6654991598525432, 0.268538386566373,
 -0.46832561733480355, -0.33003518243876184, -0.9790093948096761,
 0.9526679713446081, -0.6965839809858627, -0.6719179804484089],
 [0.35190140828471755, 0.058715545226693096, 0.38906718965284715,
 -0.678705792401814, 0.3481112218995965, -0.46321767382221474,
 0.4605471455277081, 0.6998273613113667, 0.25120457548157393,
 -0.10578382803198763, 0.1748850790921804, 0.18028906590605576,
 -0.7637759840800258, -0.197171691996172, -0.834978121634472,
 0.12374150790966493, -0.6049337003750429, 0.13417688167418831,
 0.14639492511987864, -0.520650026055211, 0.3901967466473304,
 -0.01272600468928847, -0.65803234613638, 0.8325438184563928,
 0.7015931354820373, 0.8404169699333144, 0.20582123586426843,
 465 0.9820049293146567, 0.7077041060671521, 0.18982992965485626,
 0.5358830893806832, 0.727647497011131, 0.9648864552103757,
 -0.6485861887616593, -0.47070819613814946, -0.8654532745866821,
 -0.30081605628489605, -0.5902820468422059, -0.5987965123902435,
 0.17180979286455417], [-0.6225060253108081, 0.315640201252408,
 -0.30931410859003294, -0.32894871452007896, 0.5246819153769389,
 0.7450596407706399, 0.6458227754465072, -1.0008292159421472,
 0.8007778588550597, 0.020704348046830974, 0.42698328123245993,
 -0.45299469568092743, 1.1085656664215697, -0.9414700742925355,
 -0.17389516604907954, 0.3050082928450359, -0.005328464530167936,
 0.07196829645549474, 0.6737010988281155, 0.6210755320905585,
 -0.6126723025236303, 0.17222892555640432, -0.1266136760869763,
 -0.6082963353040739, 0.32441182105353955, 0.22296772239550972,
 0.08598420317672203, 0.22890580947061923, -0.15483837469450462,
 -0.6309147140105574, -0.7512404631399754, 0.8490523672200172,
 -0.9887356184644712, -0.0639303778153393, 0.49473752366994217,
 0.604046960835961, -0.08491328249892778, 0.28955296483793014,
 0.32597222149003047, -0.4144684754450277], [1.1582853768895354,
 0.21109772067005247, -0.7158018694188236, -0.7219498583538312,
 -0.5082982220575756, -0.15116903408043564, 0.20397015514250438,
 -0.6443217780870414, 0.3812044346484128, -0.25239221694806563,
 -0.7559356415672959, 0.7275977566460975, -1.0319217349584076,
 -0.22185665406466337, -0.3621869765766397, 0.3958680665035149,
 -0.5691717897597255, -0.22574382042634822, 0.7360565188925718,
 0.5885033392728298, 0.6851182396151182, 0.6687778374910316,
 0.9698217364766895, -0.26750097370206016, -0.018323809135229158,
 -0.9395701752479648, 0.5159165016533238, 0.4523184079510531,
 -0.8550971773875436, 0.1841946268978807, 0.5049186087300309,
 -0.5114304148104705, -0.21756714910043307, -0.4804228984345346,
 -0.6320129705612818, 1.018502546589669, -0.042723304415926415,
 -0.43949283284114093, -0.10111332397308338, 0.5389322481104536],
 [0.4783171832560095, 0.12293851574695744, -0.7446424436892752,
 -0.695844826923983, 0.38278481947181325, 0.37328625402309057,
 -0.9598158979360902, -0.3391646175283339, -0.11103184515317721,
 0.33634563737398376, -0.5825130401529383, -0.7332889132542181,
 0.166439781520251, -0.9283492951613361, -0.7566826713086567,
 -0.6839063552966771, -0.3851477528163215, 0.04694288745246926,
 -0.09882241810765914, 0.5208954088422755, 0.22629687009972652,
 0.38757168251631097, 0.5328921270344196, 0.008329836193692198,
 0.8996484078765875, -0.8198120179016742, -0.8964512859338147,
 -0.7959358220254014, 0.5682779177927593, 0.4628795157848663,
 -0.08824822459407211, 0.21060747781024908, -0.7033357198506835,
 0.5616388371719717, -0.8309363150853754, 0.17567454351206122,
 -0.9283040018203461, -0.09493837415069513, 0.24536367622886504,
 0.6391874206390408], [-0.2904250607408516, -0.39502436294293064,

-0.8556681323878765, 0.020858070687454364, -0.09991532740934489,
 0.5817550767329478, 0.8031522066102985, 0.21122532608986677,
 -0.8218776145864822, 0.7661250215043425, 0.9013894452377136,
 -0.5742161149007923, -0.6592503584921262, 0.14628864794013333,
 -0.06934107523264055, 0.4622877686028598, -0.5584979838486099,
 0.6427050723827626, -0.7838516715541783, -0.06054544142813354,
 -0.5609516708686991, 0.11091554867589569, 0.7159713644360246,
 0.29084225540035225, -0.17440391627530333, -0.841385460669078,
 -0.05242076367993086, 0.6764740705298865, 0.20572650572357945,
 -0.16862142413943318, 0.6082168486520592, 0.9241406477216848,
 -0.2397213988566414, -0.18556375447010426, 0.43939651596984625,
 -0.08186617805050117, 0.5605791674002039, -0.5100779832634618,
 -0.4582486396860904, -1.0445033889691586], [0.10624977981630539,
 -0.4809768383701689, 0.4751250330786772, -0.9856883532053774,
 -0.47303428907694695, 0.3884724906155276, 0.9920592063206293,
 -1.0277624359806854, -0.15210194097494817, -1.0053280834802452,
 0.8856903942104003, -0.7288104992170115, -0.21133759270400035,
 -0.341113785008073, 0.4804928801649485, 0.07755444213051982,
 -0.5569829307514401, -0.5575509582203081, -0.6840087072915373,
 -0.9818065236024467, -0.5779573817592878, 0.6365843402881415,
 -0.3501709958743157, -0.022186449844843754, -0.4960920448320446,
 -0.6088985531152681, -0.8250905661294441, 0.7472943422661664,
 -0.7480630396661709, 0.6016675095139887, 0.2407356331959203,
 0.6294535855472739, -0.028330436374471288, 0.6946111371536204,
 0.3758624625559842, 0.5048364711412239, -0.31861636019472445,
 -0.21253144842859836, -0.052710948003140426,
 0.28079236605246516], [-0.5490851272102232, -0.7437825031197308,
 -0.09306261144279887, -0.41095289890917713, 0.8699865580975139,
 -0.6895057406166902, -0.43132843665783216, -0.5074028108017912,
 0.49569515114455126, 0.3965099232634634, 0.7076604492102069,
 -0.5899164576306264, 0.585247521061209, -0.008527705659675259,
 -0.7189222671337642, 0.719207440910141, 0.6883799190333451,
 -0.5896076793052125, 0.577480987517979, -0.35929939252870924,
 0.011050558350596068, -0.6660177895709654, 0.24140277952110478,
 0.8851460180921342, -0.7160063284315169, 0.04077043209882506,
 -0.6633951863811693, -0.1149332211011706, 0.7063949007338123,
 0.4765285685155958, -0.3151955095097356, -0.27989136017954513,
 -0.6442164548282938, -0.020059235930777273, 0.49873743838089557,
 -0.558435999749067, 0.1779008509874598, -0.8542519052152995,
 -0.3989653031196475, -0.45640184527307], [-1.21217359589114,
 -0.4504767681781005, -0.15976798279221022, -0.02710343344184992,
 0.502108389462774, 0.7303684934696778, -0.3016037070820521,
 -0.9598268835036168, 0.6845261042768884, -0.1118965691571068,
 -0.9081528625028119, -0.8660908998358295, -0.5231036301375277,
 0.5775552389028601, 0.16352826537565215, 0.37148410743924093,
 -0.2151384215927347, -1.0298816201450263, -0.048849876402495815,
 0.6049247852536758, 0.5734121311309613, -0.12413443422615163,
 0.5868987867025312, -0.024658041938186326, 0.6909912700163195,
 -0.3994495792220578, -0.7288102120168545, -0.418643874407574,
 0.7175087429370249,
 466 0.16939316906022858, 0.16064308153573068, 0.06715950149528835,
 0.345246915655312, -0.9801832974481328, -0.20615992778895587,
 0.8564273409282349, 0.608599977456359, 0.7876820532386014,
 -0.05774858447308989, 0.5965798342693593], [0.032431207244709624,
 -0.01699625306956694, -0.030512998232041774,
 -0.1980218141888685, -0.0093125026747563, 0.024474570744769067,
 0.02789958692034049, -0.0782535572991197, -0.06905362460770183,

0.024014859025046062, -0.015653018329558076,
 -0.07556943835205766, -0.028227295060639396, 0.08931732635324126,
 -0.09015323923527024, -0.022344527565196146,
 0.08792838615562495, -0.11307975639947786, 0.10373734849104468,
 -0.28659766487236515, 0.012280358376023771,
 -0.057381603226161974, 0.12618702709589544, 0.010634828107930197,
 0.10033378533041956, 0.03522919345042774, -0.1573979002806535,
 0.020873861170158214, -0.019870762693283327,
 -0.05936953076127085, -0.08543979615771786, 0.01879939143630152,
 0.03839816873205584, -0.03287123899071038,
 -0.0045291184332379095, -0.014281118369313985,
 0.11227154623924404, 0.10339348093249137, -0.003134573148018051,
 0.1141822283337402]], [[-0.6008223667442196, 0.4226392782889019,
 1.0006454052163585], [-0.5021908911983785, 0.3360724016683659,
 -0.6094066062981487], [0.6612800539606875, 0.6820943543953559,
 0.4388818368454126], [-0.8798787251880836, 0.8228537414178073,
 0.5428125660251494], [-0.14213278441066893, 0.28986586950849086,
 -0.08210894754651758], [0.5611234865247732, 0.8127898935902709,
 -0.3735973968727091], [-0.3529941628537897, -0.4299760200252432,
 -0.6812572334847985], [-0.74005331952447, 0.12746493027592515,
 0.30382917745495064], [-0.04736236831224823,
 0.0047814002410479645, 1.2012599458595383], [-0.7947183639836974,
 0.36518533505494943, 0.004075463349799456],
 [-0.08265181491511314,
 467 0.17471927634315815, 0.5385092297406808], [-0.12081077258100634,
 -0.2866426163435784, -0.08891223315277941], [0.8314675115574764,
 -0.25521798324481887, -0.7066366063260358], [0.8349750368108949,
 -0.8874574440220472, 0.14109297904878987], [0.925536714412882,
 0.45991724268296946, 0.6038844997751359], [0.06111418486805764,
 0.6820894088943827, 0.16097415085670289], [-0.8193666942432575,
 0.14100917871024216, 0.5585082153353702], [0.5760123532150492,
 1.0479711027572545, 0.5302269193889573], [0.2558258910654179,
 0.6658607328146922, -0.02197980254547798], [0.19655407106771175,
 -1.0349001256889216, 0.18332860846381566], [-0.22061748551977606,
 0.3035541404509559, 0.05725712668703925], [-0.1355387904441537,
 0.13854665313358513, -0.5315928572384069], [0.5924048607858386,
 -0.4754479516727165, 0.38540691386714704], [-0.7464095309838575,
 0.1832051279543432, 0.23102626393647438], [0.4416725426621595,
 -0.10699724059130082, 0.49432526860067416],
 [0.045226758555078336, -0.026912827969611617,
 0.18473596933824335], [-0.14435250909595737, 0.6722064496321642,
 0.06606919142516528], [0.09156962580807805, 0.8260139064138163,
 0.7573771868489236], [0.3374527549015352, -0.167315529973452,
 0.7431569718318898], [0.05980276038804148, -0.3919576033106627,
 -0.7890393180458202], [-0.5901562195133425, 0.21779953669278873,
 -0.6598031218656097], [0.5138010920462077, 0.16997887819309823,
 0.060843552295139214], [0.5709874282958407,
 468 0.1670455815744882, -0.22796001055023643], [-0.5859770151363236,
 0.26783298774838765, -0.932449594767259], [-0.8134363542273567,
 -0.22177848006711193, -0.625539872022138], [-0.38142542210521785,
 0.1568631218743948, -0.04259889859425982], [-0.3958340699274266,
 -0.4145518906955168, 0.029952963457541986], [0.1483165311476966,
 -0.021317512772147853, -0.9352160055344191],
 [0.6252679960859655, 0.12178592925381074, 0.648210867123195],
 [-0.590788269018215, -0.4969709369541748, 0.8187934142038181],
 [-0.013307591508491812, -0.13460179970697206,
 0.08399866748646273]]]